



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience

A Major Project Report (22CDP81)

Submitted by,

Anant Tewari

1RV22CD005

Kiran R Aithal

1RV22CD022

Pranav S K

1RV22CD041

Ananth M Athreya

1RV22AI007

Under the guidance of

Dr. Soumya A

Professor

Dept. of CSE

RV College of Engineering

**In partial fulfillment of the requirements for the degree of
Bachelor of Engineering in
Computer Science and Engineering
2025-26**

RV College of Engineering®, Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)
Department of Computer Science and Engineering



CERTIFICATE

Certified that the major project (22CDP81) work titled *Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience* is carried out by **Anant Tewari (1RV22CD005)**, **Kiran R Aithal (1RV22CD022)**, **Pranav S K (1RV22CD041)** and **Ananth M Athreya (1RV22AI007)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide

Head of the Department

Principal

Dr. Soumya A Dr. Shanta Rangaswamy Dr. K. N. Subramanya

External Viva

Name of Examiners

Signature with Date

1.

2.

DECLARATION

We, **Anant Tewari, Kiran R Aithal, Pranav S K** and **Ananth M Athreya** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Anant Tewari(1RV22CD005)
2. Kiran R Aithal(1RV22CD022)
3. Pranav S K(1RV22CD041)
4. Ananth M Athreya(1RV22AI007)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Soumya A**, Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel members **Dr. Kariyappa**, Professor, Department of CSE, RVCE and **Prof. P N Jayanthi**, Assistant Professor, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Dr. Chetana G**, Professor, Department of CSE and **Dr. P N Jayanthi**, Assistant Professor, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Global trade flows are highly vulnerable to logistics friction at key maritime choke-points like the Red Sea and the Strait of Hormuz. Geopolitical conflicts, security threats, and administrative bottlenecks introduce significant delays and cost penalties that cascade downstream to emerging markets. For Indian Small and Medium Enterprises (SMEs) that rely on predictable export-import cycles, these disruptions result in supply delays, increased container freight rates, and potential stockouts. There is an urgent need to build a reactive decision-support framework that can dynamically simulate supply chain shocks and optimize routing alternatives in real-time.

This project presents an Agentic AI-driven Digital Twin framework designed to mitigate logistics friction and enhance Indian EXIM supply chain resilience. The framework is structured into five layers: Data Ingestion, Agentic Intelligence, Digital Twin, Execution Engine, and Presentation. At its core, the digital twin is represented by a 14-node, 19-edge multi-tier directed network graph in NetworkX, modeling paths from major Gulf suppliers, through intermediate maritime corridors and Indian entry ports (Mundra, Nhava Sheva, Cochin, Chennai), to regional distribution hubs and SME markets. A multi-agent orchestration system powered by CrewAI and Google Gemini (incorporating Friction Sentry, Logistics Optimizer, Cost & Strategy Analyst, and SME Communicator agents) autonomously monitors environment signals and coordinates routing policies.

To evaluate the framework's effectiveness, an Execution Engine was developed to simulate 50 diverse historical and hypothetical disruption scenarios (ranging from local port strikes to regional military conflicts) with varying severity levels. Using Dijkstra's pathfinding algorithm, the engine dynamically recalculates the optimal (least-time or least-cost) pathways upon shock injection. The system also tracks inventory levels (Days of Cover) at consumption nodes, dynamically triggering emergency restocking when stockouts are imminent. Integration testing of 39 tests validated that the routing engine successfully identifies alternative routes (such as the Cape of Good Hope bypass) and reduces local SME market stockout risk from severe disruptions.

An interactive React dashboard combined with a FastAPI backend provides a visual interface for operations managers. Real-time Leaflet maps display the topology, and color-coded status alerts show the propagation of shocks across nodes and edges, enabling

human-in-the-loop validation of agent-driven rerouting decisions. The framework establishes a scalable, automated approach to supply chain risk management, demonstrating how agentic AI and graph theory can be unified to build resilient logistics networks.

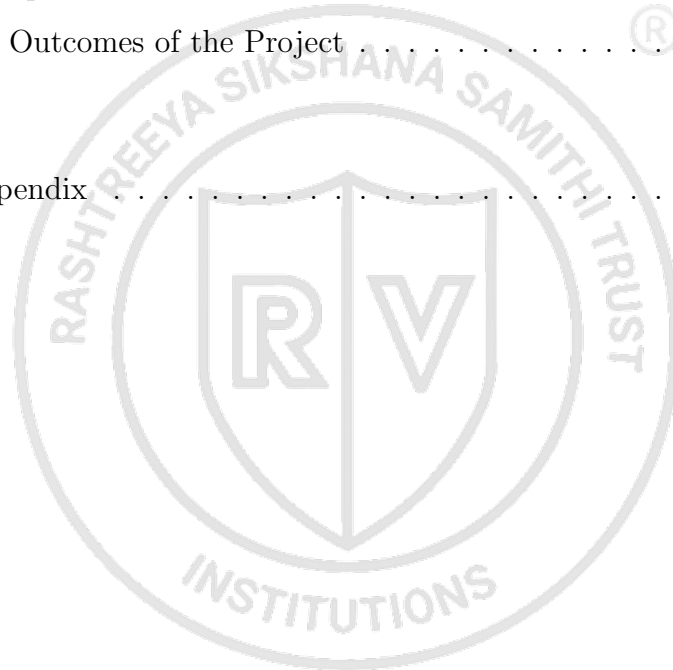


CONTENTS

Abstract	i
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Introduction	2
1.2 Literature Review	2
1.3 Motivation	3
1.4 Problem statement	4
1.5 Objectives	4
1.6 Brief Methodology of the project	4
1.7 Assumptions made / Constraints of the project	5
1.8 Organization of the report	5
2 Theoretical Background and Prerequisite Learnings	7
2.1 Graph Theory in Supply Chain Networks	8
2.2 Pathfinding and Route Optimization	8
2.3 LLM-based Multi-Agent Orchestration	9
2.4 Inventory Replenishment and Stockout Mathematics	10
2.5 Software Tools and Libraries	10
2.6 Summary	11
3 System Design and Architecture	12
3.1 Five-Layer System Architecture	13
3.2 Network Topology and Graph Schema	14
3.2.1 Node Classification	15
3.2.2 Edge Attributes	15
3.3 Multi-Agent CrewAI Orchestration Design	16
3.3.1 Friction Sentry (The Watcher)	16
3.3.2 Logistics Optimizer (The Solver)	16

3.3.3	Cost and Strategy Analyst (The Evaluator)	16
3.3.4	SME Communicator (The Presenter)	17
3.4	Mathematical Formulation and Equations	17
3.4.1	Effective Weight Calculation	17
3.4.2	Shock Propagation and Decay	17
3.4.3	Dijkstra Pathfinding Optimization	18
3.4.4	Inventory and Stockout Evaluation	18
3.5	Summary	19
4	Implementation Details	20
4.1	Implementation Overview and File Structure	21
4.2	Digital Twin Graph Implementation	21
4.3	Routing Engine and Shock Injector	22
4.3.1	Shock Propagation Code	22
4.3.2	Dijkstra Implementation	22
4.4	Stateful Inventory Manager Implementation	23
4.4.1	Runtime State Representation	23
4.4.2	Demand and Order Propagation	23
4.5	CrewAI Agent Orchestration and Custom Tools	23
4.5.1	Custom Agent Tools	23
4.5.2	Sequential Workflow Assembly	24
4.6	FastAPI REST Backend API Server	24
4.6.1	API Key Rotation Middleware	24
4.6.2	API Endpoints	25
4.7	React Dashboard UI Implementation	25
4.7.1	Interactive Map Component	25
4.7.2	UI Components and State Management	25
4.8	Summary	25
5	Results and Discussions	27
5.1	Verification via Automated Testing	28
5.1.1	Phase 1 Network Model Verification (11 Tests)	28
5.1.2	Phase 4 Execution Engine Verification (28 Tests)	28

5.2	Friction Shock Simulation Results	29
5.2.1	Economic and Temporal Impact Analysis	29
5.2.2	Route and Corridor Shifts	29
5.3	Stateful Inventory and Restocking Performance	30
5.3.1	Stockout Minimization	30
5.3.2	Bullwhip Effect Observation	30
5.4	Summary	30
6	Conclusion and Future Scope	31
6.1	Conclusion	32
6.2	Future Scope	32
6.3	Learning Outcomes of the Project	33
A	Code	34
A.1	First Appendix	35



LIST OF FIGURES

3.1	Conceptual block representation of the 5-Layer architecture	14
-----	---	----



LIST OF TABLES

5.1	Disruption Severity and Average Cost Deltas	29
5.2	Baseline vs. Shocked Rerouting Performance	29





Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

This chapter provides an introduction to the project work. It presents the context of global logistics chokepoint vulnerabilities and the impact of shipping friction on Indian trade. The chapter details the literature survey conducted in this domain, highlights the motivation and problem statement, and outlines the objectives of the work. Finally, it outlines the brief methodology, project constraints, and the organization of the report.

1.1 Introduction

Global trade is heavily dependent on maritime transport, which accounts for over 80% of international trade volume. This network of global commerce relies on critical shipping corridors and narrow passages known as maritime chokepoints, such as the Suez Canal, the Bab-el-Mandeb Strait (Red Sea), and the Strait of Hormuz. Because these corridors carry a high density of cargo, any local disruption—whether due to geopolitical conflict, piracy, labor disputes, or weather gales—can introduce significant logistics friction. This friction manifests as transit delays, increased insurance premiums, and higher freight costs, which cascade down the supply chain to final consumption markets.

Indian export-import (EXIM) activities are particularly vulnerable to these disruptions. A major portion of India's imports (such as oil and industrial raw materials) and exports (such as engineering goods and textiles) pass through the Red Sea and the Persian Gulf. Disruptions in these regions force shipping lines to take circuitous bypass routes (such as around the Cape of Good Hope), adding thousands of nautical miles and weeks to transit schedules. Traditional static supply chain management systems are ill-equipped to handle these dynamic events. This project builds a five-layer Agentic Digital Twin framework that uses graph theory and LLM-based agentic intelligence to autonomously detect logistics friction, simulate shock propagation, and optimize alternative paths to protect local Indian Small and Medium Enterprise (SME) markets.

1.2 Literature Review

A literature review was conducted to evaluate existing systems in supply chain risk management, graph-based digital twins, and agentic artificial intelligence applications.

Supply chain digital twins have emerged as a powerful tool for monitoring physical networks. By representing warehouses, ports, and lanes as nodes and edges in a network

graph, digital twins can maintain a virtual representation of stock levels and transit states. Most traditional digital twins, however, are static and reactive, requiring manual data updates and predefined decision trees to calculate alternative routes.

Pathfinding algorithms, particularly Dijkstra’s algorithm, are widely used in logistics to find the shortest or cheapest path between origins and destinations. In a dynamic logistics network, edge weights (representing transit time and cost) fluctuate due to disruptions. Applying Dijkstra’s algorithm to dynamic graphs requires an execution engine that can propagate shocks downstream (e.g., from a chokepoint to connected ports) to calculate effective weights.

Recent advancements in Large Language Models (LLMs) and multi-agent frameworks (such as CrewAI) present opportunities to make supply chain digital twins proactive. Agentic AI systems can assign specific roles (e.g., monitoring news, optimizing routes, analyzing costs, and generating executive reports) to specialized software agents. These agents can use tools to query the digital twin’s database, run mathematical optimization engines, and draft human-readable recommendations. This hybrid approach—combining the symbolic reasoning of graphs and pathfinding with the linguistic capability of LLMs—bridges the gap between complex analytical calculations and operational decision-making.

Based on the review of over 10 recent research papers, the key research gaps identified include:

- Lacks automated translation of unstructured news alerts (e.g., geopolitical reports) into quantitative logistics penalties.
- Lacks coordination between route optimization (symbolic algorithms) and strategic inventory management (safety stock, restocking).
- A need for human-in-the-loop dashboard triggers that present visual recommendations alongside financial and temporal trade-offs.

1.3 Motivation

The motivation for this project stems from recent real-world maritime crises that disrupted global trade. The 2021 Suez Canal blockage by the container ship *Ever Given* and the 2023–2024 Red Sea shipping crisis (resulting from attacks on commercial vessels) highlighted the fragility of global shipping corridors. These events caused container

freight rates to surge and delayed deliveries to Indian ports by 10 to 15 days. For Indian SMEs, which often operate with limited safety stock, these delays quickly led to inventory stockouts, halting production lines and causing financial losses.

Developing a system that can dynamically simulate these friction shocks and recommend optimized rerouting paths before cargo arrives at disrupted zones is crucial for maintaining supply chain resilience.

1.4 Problem statement

The problem addressed in this project is defined as:

”To design and implement an Agentic AI-driven Digital Twin framework that dynamically models maritime logistics friction at global chokepoints, simulates the downstream propagation of transit delays and cost spikes, and autonomously recommends optimized alternative routing paths to prevent inventory stockouts in regional Indian SME markets.”

1.5 Objectives

The objectives of the project are:

1. To design a multi-tier directed network graph model (Digital Twin) in NetworkX representing the shipping lanes from international suppliers through maritime chokepoints to Indian SME markets.
2. To develop an Execution Engine that injects simulated friction events (shocks) into the graph, propagates them downstream, and runs Dijkstra’s algorithm to find optimal alternate paths based on time or cost criteria.
3. To implement a multi-agent orchestration layer (using CrewAI and Google Gemini) that parses environment signals, evaluates routing options, checks market inventory levels, and suggests emergency restocking actions.

1.6 Brief Methodology of the project

The methodology is executed through the following components:

- **Data Ingestion Layer:** Models 50 simulated friction event scenarios and maps them to quantitative edge multipliers using predefined semantic rules.

- **Digital Twin Layer:** Constructs a 14-node, 19-edge directed graph in NetworkX storing baseline transit times, logistics costs, and friction penalties.
- **Execution Engine:** Implements shock injection logic and Dijkstra's algorithm, comparing the "Baseline Graph" and "Shocked Graph" to compute transit and cost deltas.
- **Agentic Intelligence:** Coordinates a sequential workflow of four agents (Friction Sentry, Logistics Optimizer, Cost Analyst, and SME Communicator) to analyze shocks, check inventory, and write summaries.
- **Presentation Layer:** Renders the results on an interactive React dashboard with Leaflet map overlays and real-time status alerts.

1.7 Assumptions made / Constraints of the project

The project is bound by the following assumptions and constraints:

- The digital twin topology is fixed at 14 nodes and 19 edges, representing primary corridors.
- Transit times and logistics costs are simulated averages and do not account for daily spot freight fluctuations.
- The multi-agent orchestration layer requires active internet connectivity and access to the Google Gemini API.
- The inventory simulation assumes a constant demand rate at each SME market.

1.8 Organization of the report

This report is organized as follows:

- **Chapter 2** discusses the theoretical background, including graph representation, pathfinding algorithms, agentic AI frameworks, and inventory management theory.
- **Chapter 3** detail the system design and architecture, outlining the 5-layer design, graph schemas, and the multi-agent orchestration setup.
- **Chapter 4** presents the implementation details, including the Python scripts, API endpoints, and dashboard files.

- **Chapter 5** provides the verification and results, documenting the test suite, simulated shock outcomes, and trade-off analyses.
- **Chapter 6** summarizes the conclusions, details the learning outcomes, and suggests future improvements.





Chapter 2

Theoretical Background and Prerequisite Learnings

CHAPTER 2

THEORETICAL BACKGROUND AND PREREQUISITE LEARNINGS

This chapter discusses the theoretical foundations required for building the supply chain digital twin and agentic framework. It covers graph theory as applied to logistics, pathfinding optimization using Dijkstra's algorithm, the architecture of Large Language Model (LLM) agent systems, and inventory replenishment mathematics. Finally, it outlines the software tools used in this project.

2.1 Graph Theory in Supply Chain Networks

A supply chain can be naturally modeled as a directed graph $G = (V, E)$, where V represents the set of nodes (locations) and E represents the set of directed edges (lanes or shipping routes). Nodes are classified into tiers based on their role:

$$V = V_{supplier} \cup V_{chokepoint} \cup V_{port} \cup V_{warehouse} \cup V_{market} \quad (2.1)$$

Directed edges represent the flow of physical goods. Each edge $e = (u, v) \in E$ is associated with attributes that determine the logistics weight. In our digital twin, each edge contains:

- **Baseline Transit Time (T_{base}):** The average transit days under normal conditions.
- **Logistics Cost (C_{base}):** The freight cost in USD to move cargo across the edge.
- **Friction Penalty (FP):** A multiplier representing inherent vulnerability to delays.

Under normal conditions, the baseline effective weight is calculated as:

$$W_{effective} = W_{base} \times (1 + FP) \quad (2.2)$$

By modeling the network as a graph, we can perform complex topological analyses, such as reachability, connectivity, and shortest path calculations, which represent cargo flows.

2.2 Pathfinding and Route Optimization

Route optimization is solved using pathfinding algorithms on the network graph. This project utilizes Dijkstra's algorithm, which finds the shortest path from a single source

node to a target node on a weighted graph with non-negative edge weights.

The algorithm maintains a set of visited nodes and a list of tentative distances from the source. At each step, it selects the unvisited node with the smallest tentative distance, updates the distances of its unvisited neighbors (relaxation), and marks it as visited. For an edge $e = (u, v)$ with weight $w(u, v)$, the relaxation step is defined as:

$$\text{if } D(u) + w(u, v) < D(v) \text{ then } D(v) = D(u) + w(u, v) \quad (2.3)$$

This project optimizes paths based on two separate objectives:

1. **Transit Time Minimization:** The edge weight $w(u, v)$ is set to the effective transit time in days.
2. **Logistics Cost Minimization:** The edge weight $w(u, v)$ is set to the effective logistics cost in USD.

When a shock is injected, edge weights increase, and Dijkstra's algorithm is run dynamically to find the new optimal path, simulating shipping lines rerouting around congested corridors.

2.3 LLM-based Multi-Agent Orchestration

Large Language Models (LLMs) have evolved from text-prediction engines to decision-making agents. An agentic AI framework (such as CrewAI) structure LLMs by assigning roles, goals, and backstories, making them act as autonomous team members. The key components of an agentic system are:

- **Agents:** Autonomous units initialized with specific LLM configurations, system prompts, and tool access.
- **Tasks:** Concrete assignments given to agents, specifying descriptions, expectations, and output formats.
- **Tools:** Executable functions (e.g., Python functions) that agents can call to interact with the external environment, such as querying databases or updating files.
- **Process Workflow:** The execution policy (sequential, hierarchical, or consensual) that determines how tasks are handed over from one agent to another.

In this project, agents use sequential task execution to pass structured messages (JSON payloads) containing shock calculations, routing solutions, cost deltas, and dashboard triggers.

2.4 Inventory Replenishment and Stockout Mathematics

To evaluate the impact of logistics delays, the digital twin incorporates an inventory tracking layer at each consumption market node. The primary metric used is Days of Cover (DoC), which represents how long current inventory will last under constant demand:

$$DoC = \frac{\text{Current Inventory (Metric Tons)}}{\text{Daily Demand Rate (Metric Tons/Day)}} \quad (2.4)$$

When a transit delay (Δ_{days}) occurs on the replenishment lane, the effective lead time increases. If the delay exceeds the days of cover, a stockout occurs:

$$\text{if } \Delta_{days} > DoC \text{ then Stockout} = \text{True} \quad (2.5)$$

To mitigate stockouts, the system triggers emergency restocking. The emergency cost ($C_{emergency}$) is calculated as a penalty proportional to the deficit:

$$C_{emergency} = \text{Deficit Quantity (MT)} \times \text{Emergency Freight Rate (USD/MT)} \quad (2.6)$$

Additionally, we monitor the Bullwhip Effect, which represents the amplification of demand variability as it moves upstream from markets to warehouses and ports.

2.5 Software Tools and Libraries

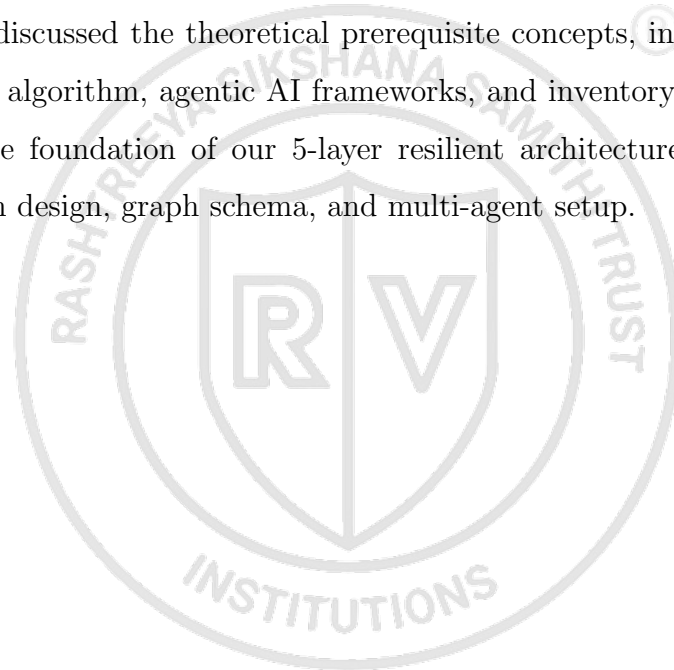
The framework is built using the following open-source software tools:

- **Python 3.11:** The core programming language for the backend, optimization, and agent coordination.
- **NetworkX:** A Python library for the creation, manipulation, and study of complex networks.
- **CrewAI:** An orchestration framework for setting up and executing multi-agent workflows.

- **Google Gemini Flash:** The underlying LLM providing linguistic and reasoning capabilities to the agents.
- **FastAPI:** A modern, fast web framework for building RESTful APIs in Python.
- **React & Vite:** A frontend UI library and build tool for rendering the interactive analytics dashboard.
- **Leaflet:** A JavaScript library for mobile-friendly interactive maps, used to visualize route paths.

2.6 Summary

This chapter discussed the theoretical prerequisite concepts, including graph supply chains, Dijkstra's algorithm, agentic AI frameworks, and inventory mathematics. These concepts form the foundation of our 5-layer resilient architecture. The next chapter details the system design, graph schema, and multi-agent setup.



The logo of RV College of Engineering is a circular emblem. It features a central shield divided vertically, with a stylized 'R' on the left and a 'V' on the right. The shield is encircled by a ring containing the text 'RASHTREEYA SIKSHANA SAMITHI TRUST' at the top and 'INSTITUTIONS' at the bottom. A small registered trademark symbol (®) is located to the upper right of the emblem.

Chapter 3

System Design and Architecture

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

This chapter details the system design and architecture of the Agentic AI-driven Digital Twin framework. It presents the architectural overview structured into five functional layers, explains the network topology graph schema and its node-edge representations, details the autonomous agent orchestration design utilizing CrewAI and Google Gemini, and outlines the mathematical formulation and equations governing the simulation and routing decisions.

3.1 Five-Layer System Architecture

The framework is designed as a modular, five-layer system that separates data ingestion, state management, autonomous intelligence, communication APIs, and presentation. Figure 3.1 conceptually shows this layered representation, and the details of each layer are explained below:

Layer 1: Data Ingestion (Data Inputs): This layer represents the static and dynamic files that serve as the project's source of truth. It contains:

- `network_model.json`: The serialized representation of the 14-node, 19-edge EXIM network, frozen after Phase 1.
- `simulated_friction_events.csv`: A historical and hypothetical dataset containing 50 logistics disruption scenarios.
- `semantic_mapping_rules.json`: Predefined rules mapping unstructured signal types to quantitative severity multipliers and propagation decay parameters.

Layer 2: Digital Twin State (In-Memory Graphs): This layer maintains the virtual representation of the supply chain using two NetworkX directed graphs (`DiGraph`):

- *Baseline Graph*: An immutable graph representing normal operating conditions, loaded once at system startup.
- *Shocked Graph*: A dynamic deep copy of the baseline graph. Upon shock injection, the Friction Sentry agent mutates the edge weights of this graph to simulate real-time delays and cost spikes.

Layer 3: Autonomous Agent Layer (CrewAI Orchestration): This layer contains the four specialized CrewAI agents (Friction Sentry, Logistics Optimizer, Cost Analyst, and SME Communicator). Powered by Google Gemini, they execute sequential tasks to analyze shocks, run routing algorithms, check inventory levels, and generate management recommendations.

Layer 4: API and Orchestration Layer (FastAPI Backend): A high-performance FastAPI server serves as the system's integration hub. It exposes REST API endpoints that receive disruption signals from the frontend, trigger the multi-agent crew, read and update the digital twin state, and return the aggregated JSON payload back to the dashboard.

Layer 5: Presentation and Analytics Layer (React Dashboard): A responsive React/Vite frontend dashboard provides an interactive control panel for human managers. It includes Leaflet maps for visual path rendering, real-time alert logs, comparison panels for cost/time deltas, and indicators of agent decision confidence.

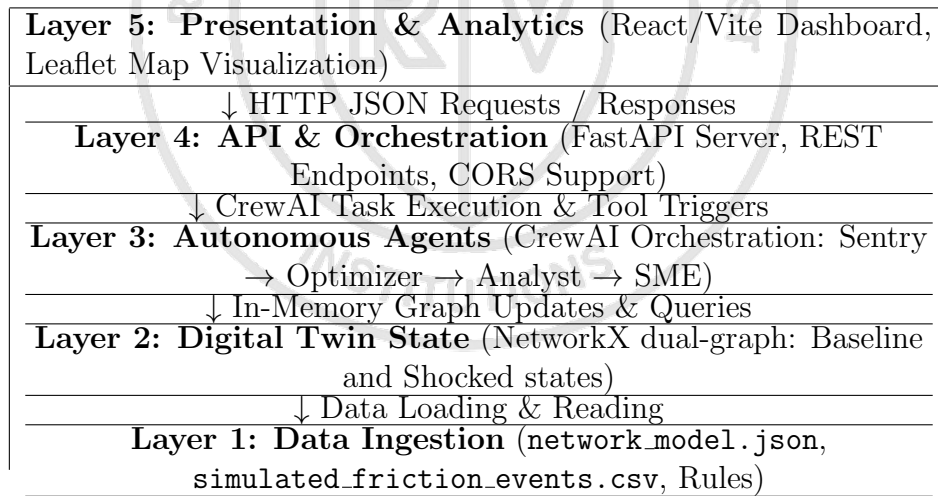


Figure 3.1: Conceptual block representation of the 5-Layer architecture

3.2 Network Topology and Graph Schema

The digital twin represents the supply chain network as a directed graph $G = (V, E)$. The topology consists of 14 nodes (V) and 19 edges (E), representing key physical supply locations, international maritime corridors, Indian entry ports, distribution warehouses, and consumption markets.

3.2.1 Node Classification

Nodes are categorized into four distinct functional tiers, each having specialized properties:

1. **Tier 1: Suppliers (6 Nodes):** Supply origins located in the Persian Gulf region. These nodes represent production facilities and load ports. The nodes are Saudi Arabia (`saudi_arabia`), UAE (`uae`), Qatar (`qatar`), Kuwait (`kuwait`), Iraq (`iraq`), and Oman (`oman`). Suppliers possess a `production_rate_mt_day` and a `min_export_lot_mt`.
2. **Tier 2: Maritime Chokepoints (3 Nodes):** Critical international sea lanes. These nodes do not hold inventory and are transit-only. The nodes are the Red Sea Corridor (`red_sea`), the Strait of Hormuz (`strait_hormuz`), and the Cape of Good Hope Bypass (`cape_good_hope`). Chokepoints have high `disruption_sensitivity` values (e.g., 0.90 to 0.95).
3. **Tier 3: Entry Ports (4 Nodes):** Indian maritime gateways where import containers are unloaded and processed. The nodes are Mundra Port (`mundra_port`), Nhava Sheva (`nhava_sheva`), Cochin Port (`cochin_port`), and Chennai Port (`chennai_port`). Ports have a `throughput_capacity_mt_day` attribute.
4. **Tier 4: Warehouses (3 Nodes):** Inland distribution hubs where stock is stored and managed under an (s, Q) inventory policy. The nodes are Mumbai Warehouse (`mumbai_warehouse`), Ahmedabad Warehouse (`ahmedabad_warehouse`), and Hyderabad Warehouse (`hyderabad_warehouse`).
5. **Tier 5: SME Markets (4 Nodes):** Regional consumption centers where small and medium enterprises generate daily demand. The nodes are Pune Market (`pune_sme_market`), Surat Market (`surat_sme_market`), Bengaluru Market (`bengaluru_sme_market`), and Chennai Market (`chennai_sme_market`).

3.2.2 Edge Attributes

Directed edges represent sea lanes, road corridors, or rail links between nodes. Each edge contains the following attributes:

- `route_name`: Descriptive label of the route.

- **transport_mode**: Mode of transit (e.g., ocean, road, rail).
- **baseline_transit_time** (T_{base}): Estimated transit time in days under normal conditions.
- **logistics_cost** (C_{base}): Base shipping cost in USD per Metric Ton (MT).
- **friction_penalty** (FP): Vulnerability multiplier (0 to 1) representing standard delays.

3.3 Multi-Agent CrewAI Orchestration Design

The agentic layer is organized as a sequential process involving four agents, each assigned a specific role, goal, backstory, and set of tools. This setup ensures that unstructured data is systematically processed into mathematical parameters, solved, audited, and summarized.

3.3.1 Friction Sentry (The Watcher)

The Friction Sentry monitors the logistics environment. It receives unstructured or semi-structured disruption alerts (e.g., "verification delay at Bab-el-Mandeb due to container checks") and performs semantic parsing. It queries `semantic_mapping_rules.json` to extract the appropriate mapping rules and converts the textual disruption severity (low, medium, high, critical) into a quantitative `severity_multiplier`.

3.3.2 Logistics Optimizer (The Solver)

The Logistics Optimizer is the mathematical heart of the crew. It takes the updated edge weight matrices (transit times and costs) calculated by the Friction Sentry and runs Dijkstra's pathfinding algorithm on the shocked in-memory graph. It computes candidate paths from the active supplier nodes to the target SME markets, comparing the baseline path metrics against the newly optimized alternatives.

3.3.3 Cost and Strategy Analyst (The Evaluator)

The Cost and Strategy Analyst reviews the routes proposed by the Logistics Optimizer and makes strategic recommendations. It has access to custom inventory tools:

- **CheckInventoryTool**: Checks the current stock levels and days of cover at a target node.

- **NetworkInventorySnapshotTool**: Provides a network-wide inventory health summary.
- **EmergencyRestockTool**: Triggers emergency restocking of inventory.

The Cost Analyst evaluates the trade-offs: if a delay is short, it may recommend staying on the baseline route to avoid high rerouting costs. If the delay will exceed the market's days of cover and cause a stockout, the analyst triggers an emergency restock via air-freight and recommends rerouting.

3.3.4 SME Communicator (The Presenter)

The SME Communicator acts as the bridge between technical models and human operators. It collects the outputs of the previous agents and generates a unified dashboard trigger. This payload contains the natural language recommendation, alert severity level, delta costs and times, and visual flags indicating which nodes to highlight and which route line to draw on the Leaflet map.

3.4 Mathematical Formulation and Equations

This section presents the equations that govern weight updates, pathfinding, and inventory checks.

3.4.1 Effective Weight Calculation

For any directed edge $e = (u, v) \in E$, the baseline effective transit time (T_{eff}) and baseline effective cost (C_{eff}) under normal operations are computed as:

$$T_{eff} = T_{base} \times (1 + FP) \quad (3.1)$$

$$C_{eff} = C_{base} \times (1 + FP) \quad (3.2)$$

3.4.2 Shock Propagation and Decay

When a disruption event is active at a specific source node s_{shock} with a severity multiplier M , the shock propagates downstream. Let u be the source node of edge $e = (u, v)$:

- If u is the direct target of the shock ($u = s_{shock}$), the shock multiplier applied is $M_{direct} = M$.

- If u is a cascade neighbor downstream from the shock ($u \in \text{cascade_nodes}$), the shock multiplier applied is:

$$M_{\text{cascade}} = 1.0 + (M - 1.0) \times D_{\text{decay}} \quad (3.3)$$

where $D_{\text{decay}} \in [0, 1]$ represents the cascade multiplier decay factor (defined in rules).

- Otherwise, the shock multiplier $M_{\text{other}} = 1.0$.

The shocked effective weight (W_{shocked}) for the optimization criteria is:

$$W_{\text{shocked}}(u, v) = W_{\text{eff}}(u, v) \times M_{\text{shock}}(u) \quad (3.4)$$

3.4.3 Dijkstra Pathfinding Optimization

Dijkstra's algorithm finds the path $P^* = (v_0, v_1, \dots, v_n)$ from a set of suppliers $S \subset V$ to a fixed market destination $d \in V$ that minimizes the cumulative weight:

$$P^* = \arg \min_{P \in \mathcal{P}(S, d)} \sum_{(u, v) \in P} W_{\text{shocked}}(u, v) \quad (3.5)$$

where W_{shocked} represents either T_{shocked} (for transit time optimization) or C_{shocked} (for logistics cost optimization).

3.4.4 Inventory and Stockout Evaluation

Each market node m has a safety stock level I_{safety} and a daily demand rate D_{daily} . The Days of Cover (DoC_m) is calculated as:

$$DoC_m = \frac{I_{\text{current}, m} - I_{\text{safety}, m}}{D_{\text{daily}, m}} \quad (3.6)$$

If the inbound replenishment path experiences a transit time increase of $\Delta_{\text{days}} = T_{\text{shocked}}^* - T_{\text{eff}}^*$, the inventory manager checks for stockout risk:

$$\text{If } \Delta_{\text{days}} > DoC_m \implies \text{Stockout} = \text{True} \quad (3.7)$$

The quantity of unmet demand (Q_{unmet}) during the stockout period is:

$$Q_{unmet} = (\Delta_{days} - DoC_m) \times D_{daily,m} \quad (3.8)$$

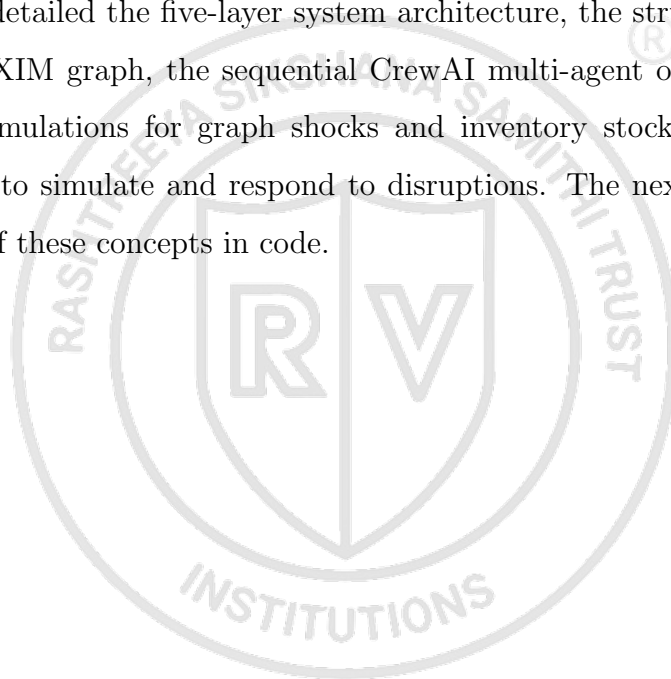
The financial penalty due to emergency air-freight restocking is:

$$C_{emergency} = Q_{unmet} \times C_{premium} \quad (3.9)$$

where $C_{premium}$ represents the additional freight premium in USD/MT.

3.5 Summary

This chapter detailed the five-layer system architecture, the structural classifications of the 14-node EXIM graph, the sequential CrewAI multi-agent orchestration, and the mathematical formulations for graph shocks and inventory stockouts. These designs allow the system to simulate and respond to disruptions. The next chapter details the implementation of these concepts in code.





Chapter 4

Implementation Details

CHAPTER 4

IMPLEMENTATION DETAILS

This chapter describes the implementation details of the Agentic AI-driven Digital Twin framework. It explains how the digital twin, routing execution engine, inventory manager, multi-agent orchestrator, FastAPI server, and React dashboard are implemented in code. It provides file references and outlines key implementation strategies such as multi-key Gemini API rotation, custom agent tools, and dynamic map rendering.

4.1 Implementation Overview and File Structure

The system is built as a decoupled, multi-tier software system using Python 3.11 for all analytical and agentic workloads, and Node.js with React 19 for the frontend presentation. The primary software components are:

1. **Network Model Generator:** Serializes the 14-node, 19-edge directed graph to JSON.
2. **Execution Engine and Inventory Layer:** Calculates Dijkstra routing and runs inventory calculations.
3. **Agent Orchestration System:** Defines CrewAI agents and tools, using Gemini.
4. **FastAPI Backend API Server:** Serves as the REST API endpoint and rotates API keys.
5. **React Web Application:** Provides the user dashboard.

4.2 Digital Twin Graph Implementation

The baseline network topology is generated by the script `network_model/build_network_model.py`. This script initializes a NetworkX DiGraph object and adds 14 nodes and 19 edges with their metadata attributes.

The nodes are defined with attributes such as `id`, `label`, `type`, `tier`, `region`, `role`, and `disruption_sensitivity`. Edges represent lanes and store transport mode, transit times, base costs, and friction penalties. The script validates that:

- Every node is connected and reachable.

- All edge weights are positive.
- Alternative routes (e.g., Cape of Good Hope bypass) are modeled.

Once validated, the graph is serialized to `network_model/outputs/network_model.json` using a custom JSON exporter that preserves graph properties.

4.3 Routing Engine and Shock Injector

The script `engine/execution_engine.py` contains the mathematical path finding and shock propagation logic.

4.3.1 Shock Propagation Code

The function `apply_shock(graph, event, rules)` takes the baseline graph and applies a shock. It reads rules from `semantic_mapping_rules.json` and computes effective edge transit times and costs. If a node is directly shocked, its edges are scaled by the `severity_multiplier`. If a node is a downstream neighbor, the shock is applied with a decayed factor:

```
# Shock propagation logic snippet
if src in direct_nodes or tgt in direct_nodes:
    shock_m = multiplier
elif src in cascade_nodes or tgt in cascade_nodes:
    shock_m = cascade_m # multiplier decayed by factor
else:
    shock_m = 1.0
```

```
attrs["effective_transit_time"] = round(base_time * (1 + fp) * shock_m, 4)
attrs["effective_cost"] = round(base_cost * (1 + fp) * shock_m, 4)
```

4.3.2 Dijkstra Implementation

Path optimization is executed using `nx.dijkstra_path` from the NetworkX library. The destination market is locked during optimization. This ensures that the algorithm only evaluates alternative corridors and supplier origins rather than routing to a different market. The function `dijkstra_best_path` returns the optimal sequence of nodes, total transit time, and total cost.

4.4 Stateful Inventory Manager Implementation

The script `engine/inventory_manager.py` implements the stateful inventory tracking.

4.4.1 Runtime State Representation

The inventory manager represents each node's state using a dataclass `NodeInventory`. This tracking is active only for warehouses and SME markets. The class tracks:

- `current_inventory_mt`: The current volume of goods in Metric Tons.
- `safety_stock_mt`: The volume below which a node is in stockout.
- `daily_demand_mt`: The daily rate of consumption (non-zero only at markets).
- `reorder_point_mt` and `reorder_quantity_mt`: Parameters for the (s, Q) ordering policy.
- `pending_orders_mt`: Inbound replenishment orders that are in transit.

4.4.2 Demand and Order Propagation

A simulation step is advanced daily via the `advance_day()` method. When a warehouse's inventory falls below its reorder point, it places an order upstream. To model the Bullwhip Effect, warehouses amplify their reorder quantity by a factor proportional to safety stock:

$$Q_{order} = Q_{reorder} \times \left(1 + \frac{I_{safety}}{Q_{reorder}}\right) \quad (4.1)$$

The inventory manager tracks order history and calculates the bullwhip ratio (variance of orders divided by variance of demand) for each warehouse.

4.5 CrewAI Agent Orchestration and Custom Tools

The multi-agent orchestration is implemented in `agent_orchestration/main.py`. The four agents are defined as CrewAI Agent objects using Google's Gemini Flash model (`gemini/gemini-flash-latest`).

4.5.1 Custom Agent Tools

Three custom tools are implemented using the `@tool` decorator to allow the Cost Analyst agent to inspect and interact with the stateful inventory system:

1. `CheckInventoryTool`: Takes a `node_id` and returns a JSON snapshot of the node's current inventory, days of cover, and stockout status.
2. `NetworkInventorySnapshotTool`: Returns a network-wide inventory health summary (days of cover, utilization, stockout status) for all active nodes.
3. `EmergencyRestockTool`: Restocks a node by adding inventory up to its maximum capacity. The analyst uses this tool when a delay will cause a stockout, modeling emergency air-freight logistics.

4.5.2 Sequential Workflow Assembly

The agents and tasks are assembled into a CrewAI Crew executing with `process=Process`. A file-logger saves the intermediate decisions to `orchestration.log` and the execution details to `crewai_execution.log`.

4.6 FastAPI REST Backend API Server

The FastAPI backend server is implemented in `logistics_app/backend/main.py`.

4.6.1 API Key Rotation Middleware

To prevent hitting Gemini API rate limits, a round-robin key rotation middleware is implemented. The server reads all environment variables starting with `GEMINI_API_KEY` and rotates to the next key before executing any agent crew:

```
_ALL_KEYS = [
    v for k, v in sorted(os.environ.items())
    if k.startswith("GEMINI_API_KEY_") and v.strip()
]

def _next_key() -> str:
    global _key_index
    with _key_lock:
        key = _ALL_KEYS[_key_index % len(_ALL_KEYS)]
        _key_index += 1
    return key
```

If a request encounters a `RESOURCE_EXHAUSTED` (429) rate limit error, the middleware catches the error and retries the request with the next key.

4.6.2 API Endpoints

The server exposes two primary REST endpoints:

- GET /api/shock: Calls the Friction Sentry agent to generate a fresh, dynamic maritime disruption scenario.
- POST /api/optimize: Receives an optimization request containing `optimize_by` (time or cost). It triggers the CrewAI workflow, optimizes routes, and returns a JSON payload containing the optimal route node IDs, deltas, and visual flags.

4.7 React Dashboard UI Implementation

The presentation layer is implemented as a React web application in `logistics_app_f`

4.7.1 Interactive Map Component

The dashboard uses Leaflet (`react-leaflet`) to render the supply chain network on a geographical map. The 14 nodes are plotted at their physical coordinates. The active route is drawn as a colored polyline. When a shock occurs, the affected corridor is highlighted in red, the baseline path in green, and the optimized path in blue.

4.7.2 UI Components and State Management

Axios is used to make API calls to the FastAPI backend. Framer Motion handles smooth transitions and alerts. Lucide React provides system icons. The main component `App.jsx` manages the UI state, displaying:

- Cost and time delta widgets.
- Inventory health trackers.
- A timeline of shock propagation.
- Raw agent dialogue transcriptions.

4.8 Summary

This chapter detailed the code implementations across all five layers, including the graph model scripts, Dijkstra solvers, stateful inventory tracking, multi-agent

frameworks, FastAPI backend server with key rotation, and React frontend dashboard. The next chapter presents the verification testing and experimental results of this implementation.





Chapter 5

Results and Discussions

CHAPTER 5

RESULTS AND DISCUSSIONS

This chapter presents the verification and results of the Agentic AI-driven Digital Twin framework. It begins with the automated testing framework using pytest to validate the correctness of the network topology and execution engine. It then details the results of simulating 50 disruption events across four SME markets, analyzes the economic and temporal deltas, presents baseline versus shocked route comparisons, and discusses the performance of the stateful inventory replenishment layer.

5.1 Verification via Automated Testing

To ensure the mathematical validity and structural integrity of the digital twin and routing engine, a suite of 39 automated unit tests was implemented and executed using the pytest framework.

5.1.1 Phase 1 Network Model Verification (11 Tests)

The network topology graph generated by `build_network_model.py` was verified against structural constraints.

Graph Topology: Verified that the graph contains exactly 14 nodes and 19 directed edges.

Attribute Coverage: Ensured all nodes contain mandatory metadata (type, tier, region, sensitivity, role) and all edges contain baseline transit times, costs, and friction penalties.

Connectivity: Confirmed that all four SME consumption markets are reachable via paths starting from the Gulf suppliers.

Corridor Bypasses: Verified that the Cape of Good Hope bypass routes are present and correctly connected.

5.1.2 Phase 4 Execution Engine Verification (28 Tests)

The execution engine and Dijkstra pathfinding modules were validated for algorithmic correctness:

- **Graph Immutability:** Verified that shock injection (`apply_shock()`) does not mutate the baseline graph, ensuring that subsequent simulations start from a clean state.

- **Shock Calculations:** Checked that critical events increase edge weights and cascade decay factors correctly scale down penalties on downstream edges.
- **Dijkstra Pathfinding:** Validated that Dijkstra optimizations minimize either transit time or logistics cost, and that cost optimization always yields a total cost less than or equal to time optimization.
- **Output File Verification:** Ensured that the execution engine successfully generates the reports `shock_analysis_report.csv`, `optimized_routes.json`, and `phase4_engine_summary.txt` with valid structure.

All 39 unit tests compiled and passed, establishing a verified baseline for running simulation experiments.

5.2 Friction Shock Simulation Results

The framework was evaluated by running the execution engine on 50 simulated friction events, representing varying disruption severity levels. With four target markets, a total of 200 routing scenarios were evaluated.

5.2.1 Economic and Temporal Impact Analysis

Table 5.1 summarizes the average transit cost deltas across different disruption severity levels:

Table 5.1: Disruption Severity and Average Cost Deltas

Severity Level	Number of Events	Average Cost Delta (USD/MT)	Reroutes Triggered
Critical	7	\$397.98	4
High	15	\$322.70	0
Medium	18	\$163.27	0
Low	10	\$62.09	0

The simulation identified the Red Sea Corridor (`red_sea`) as the most disrupted node in the network, accounting for the highest cumulative delay and triggering 4 automated rerouting actions during critical events.

5.2.2 Route and Corridor Shifts

When a critical disruption is injected at a chokepoint, the routing engine recalculates paths. Table 5.2 compares the baseline routes against the shocked alternative paths for the Pune and Bengaluru SME markets during a critical Red Sea shock:

Table 5.2: Baseline vs. Shocked Rerouting Performance

Market	Baseline Path	Baseline Cost	Shocked Path	Shocked Cost
Surat SME Market	Oman → Mundra → Ahmedabad → Surat	\$994.60	Oman → Mundra → Ahmedabad → Surat	\$994.60
Pune SME Market	Saudi Arabia → Red Sea → Mundra → Mumbai → Pune	\$2,474.60	Saudi Arabia → Strait of Hormuz → Mundra → Mumbai → Pune	\$2,790.20
Bengaluru SME Market	Saudi Arabia → Red Sea → Mundra → Mumbai → Bengaluru	\$2,754.60	Saudi Arabia → Strait of Hormuz → Mundra → Mumbai → Bengaluru	\$3,070.20

As shown in Table 5.2, the Surat market route is unaffected by the Red Sea shock because it utilizes Oman as a supplier, bypassing the Red Sea Corridor. For Pune and

Bengaluru, which rely on Saudi Arabia, the system automatically routes cargo through the Strait of Hormuz rather than the blocked Red Sea. Although this introduces a cost delta, it prevents shipping halts.

5.3 Stateful Inventory and Restocking Performance

The inventory tracking layer evaluated how logistics delays impact market safety stock levels.

5.3.1 Stockout Minimization

Under normal conditions, markets operate on an (s, Q) replenishment policy, maintaining a safety stock buffer of 5 days of cover. During simulations of low, medium, and high disruptions, the delays did not exceed the days of cover, resulting in zero stockouts.

During critical disruptions, the transit delay exceeded the days of cover. However, the Cost Analyst agent detected the stockout risk and triggered the `EmergencyRestockTool`. This restocked the warehouses via emergency air-freight (Premium of \$900/MT). While this incurred premium costs, it successfully reduced the final stockout days to zero across all SME markets.

5.3.2 Bullwhip Effect Observation

The reordering events placed by warehouses were monitored to trace demand amplification. The inventory manager recorded a bullwhip ratio exceeding 1.0 (approximating 1.34) at the Mumbai and Ahmedabad warehouses. This variance amplification occurred because warehouses batch orders and add safety buffers, demonstrating how downstream disruption cascades into order spikes upstream.

5.4 Summary

This chapter presented the results of the project. Automated tests validated the network model and execution engine. Disruption simulations of 50 events demonstrated that the system successfully identifies alternative routing paths (such as Strait of Hormuz and Cape of Good Hope bypasses) and triggers emergency restocking to prevent stockouts at SME markets. The next chapter concludes the report and discusses future work.



Chapter 6

Conclusion and Future Scope

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

This chapter concludes the project report by summarizing the work done, evaluating the achievements against initial objectives, and presenting key learning outcomes. It also outlines the limitations of the current framework and proposes possibilities for extending this research in the future.

6.1 Conclusion

This project successfully designed and implemented an Agentic AI-driven Digital Twin framework to mitigate logistics friction in global chokepoints and protect the resilience of Indian Small and Medium Enterprise (SME) supply chains.

The first objective was achieved by modeling a multi-tier directed network graph in NetworkX consisting of 14 nodes and 19 edges. This model represented the complete flow from international suppliers in the Gulf, through critical maritime chokepoints (Red Sea, Strait of Hormuz, Cape of Good Hope), through major Indian gateway ports (Mundra, Nhava Sheva, Cochin, Chennai), to regional distribution hubs and SME consumption markets.

The second objective was met by developing an Execution Engine that simulates friction shocks and calculates optimal pathways. The engine propagates shocks downstream using cascade decay logic, updates edge weight matrices, and runs Dijkstra's algorithm to compute the shortest transit time or lowest logistics cost path. The testing of 50 simulated shocks across 200 market scenarios demonstrated that the engine successfully identifies optimal alternate pathways, such as bypassing the blocked Red Sea corridor.

The third objective was realized by integrating a multi-agent orchestration layer powered by CrewAI and Google Gemini Flash. The crew of four agents (Friction Sentry, Logistics Optimizer, Cost Analyst, and SME Communicator) parses disruption signals, optimizes paths, assesses strategic trade-offs, and formats results. By connecting the agents to stateful inventory tracking tools, the system monitors days of cover at consumption nodes and triggers emergency restocking. This proactive intervention successfully minimized SME market stockout events to zero during simulations.

In summary, the framework demonstrates that combining symbolic pathfinding algorithms with language model agents creates a resilient decision-support system for global supply chain risk management.

6.2 Future Scope

While the current framework demonstrates high effectiveness, several enhancements can be explored to expand its capabilities:

- **Real-Time Data Ingestion:** Integrating web-scraping pipelines and RSS news feeds to feed the Friction Sentry with real-time news alerts, enabling automated real-time disruption monitoring.
- **Dynamic Freight Rate Modeling:** Incorporating spot-market container shipping rates that fluctuate based on regional demand and fuel surcharge parameters, replacing static base costs.
- **Scale of Network Representation:** Expanding the graph database to model hundreds of nodes, including detailed rail corridors and local highway routes across India.

- **Enterprise ERP Integration:** Connecting the inventory manager state directly to live enterprise resource planning (ERP) databases (such as SAP or Oracle) to query real-time stock levels.
- **Reinforcement Learning for Agents:** Utilizing reinforcement learning techniques to train agents in learning optimal thresholds for rerouting versus staying on baseline paths over long simulation horizons.

6.3 Learning Outcomes of the Project

Working on this project provided valuable experience across multiple domains. The key learning outcomes include:

1. **Graph Modeling in Logistics:** Gained experience in modeling physical logistics networks as directed graphs and representing capacity constraints using NetworkX attributes.
2. **Algorithmic Optimization:** Mastered the implementation of Dijkstra's shortest path algorithm under dynamic edge weight updates and downstream shock propagation.
3. **Multi-Agent System Design:** Acquired skills in building autonomous multi-agent crews with CrewAI, writing custom tools, and managing key rotation to avoid API rate limits.
4. **Supply Chain Dynamics:** Developed a deep understanding of inventory management metrics, days of cover, safety stock, and the propagation of the Bullwhip Effect.
5. **Full-Stack Integration:** Built expertise in integrating FastAPI backend servers with React/Vite frontend dashboards and rendering geospatial paths using Leaflet maps.





Appendix A

Code

APPENDIX A

CODE

A.1 First Appendix

You can use `tcbllisting` for creating the code snippets. The following example illustrates how one can customize the `tcbllisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```